

# PizzaFlow

*Build a Full-Stack AI Ordering System From PRD to Production*

Teams of 3–4 · June 23 → July 2 · 3 Stages · 100 Points

*You are the founding engineering team of SliceMatic — a new pizza delivery outlet in Delhi. They have a kitchen, riders, and a menu. You have skills, AI tools, and three weeks. Your job: design the business, build the product, and ship it live.*

**Three stages. Three deadlines. One final demo.**

| STAGE 1                       | STAGE 2                       | STAGE 3                      |
|-------------------------------|-------------------------------|------------------------------|
| PRD + Business Analysis       | Working MVP on Gradio         | Full-Stack App — Live Demo   |
| Due: June 25<br><b>20 pts</b> | Due: June 27<br><b>30 pts</b> | Due: July 2<br><b>50 pts</b> |

## 01 · THE BRIEF — WHAT YOU ARE BUILDING

### SliceMatic Ordering System

SliceMatic is a new single-outlet pizza delivery brand launching in New Ashok Nagar, Delhi. They currently take orders over the phone. The owner wants a digital ordering system that handles customer intake, menu selection, pricing, discounts, GST, and payment — and saves every order for business analysis. Over three stages you will design, prototype, and productionise this system using the full FDE stack.

Using AI tools to help you build is **expected and encouraged** — vibe coding is a core skill.

*What will be tested is whether you truly understand what you shipped. During the Stage 3 demo, any team member may be asked to explain any part of the system live.*

### Menu Data Files — Industry-Grade Format

Three .txt files are provided with your assignment package. Each line follows the format: ID;Name;Price(INR). Your program must load these at runtime — no hardcoding menu items in code. The grader will swap the files before evaluation.

| File                  | Items                                                                                                                                                  | Price Range     | Format        |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|---------------|
| Types_of_Base.txt     | Thin Crust, Thick Crust, Cheese Burst, Whole Wheat, Multigrain                                                                                         | Rs.149 – Rs.229 | ID;Name;Price |
| Types_of_Pizza.txt    | Margherita, Chicago Deep Dish, Greek Mediterranean, Farm House, Pepperoni Classic, BBQ Chicken, Paneer Tikka, California Veggie                        | Rs.299 – Rs.379 | ID;Name;Price |
| Types_of_Toppings.txt | Black Olives, Extra Cheese, Mushrooms, Green Peppers, Jalapenos, Sun-Dried Tomatoes, Caramelised Onions, Sweet Corn, Roasted Garlic, Peri-Peri Drizzle | Rs.39 – Rs.69   | ID;Name;Price |

**IMPORTANT:** The grader will replace these files with a different menu before evaluating your submission. If your code crashes when file contents change, you lose all file-loading marks. Write defensive parsing — strip whitespace, handle missing fields, validate price is numeric.

## STAGE 1

### PRD + Business Unit Economics

Due: Sunday, June 25 · 20 Points · Submit as PDF or Notion link

#### Part A — Product Requirements Document

Before writing a single line of code, think like a product manager. Your PRD must cover every requirement precisely enough that a developer who has never seen the spec could build it correctly.

- **Product Vision** — One-paragraph statement of what SliceMatic's ordering system does, who it serves, and what problem it replaces.
- **Functional Requirements** — Complete list — customer onboarding, quantity selection, menu browsing, pricing engine, discount logic, GST calculation, payment flow, order persistence.
- **Non-Functional Requirements** — Input validation rules, error messages, edge-case handling, data format for the orders log, graceful failure modes.
- **User Flow Diagram** — A complete flowchart of the ordering journey — from launch to order confirmation. Include every decision node and error branch.
- **Drawbacks Analysis** — What are the real limitations of this system as specified? What breaks when you have 1,000 orders? What is missing for a real production deployment?
- **Cost vs Value Analysis** — Estimated build effort (hours) vs measurable value to the outlet. What does SliceMatic gain — in operational efficiency, data, and customer experience?

#### Part B — Business Unit Economics

Create a realistic financial model for SliceMatic. Numbers must be internally consistent and grounded in Tier-1 Indian market realities. A reference set of numbers is provided in the companion document `SliceMatic_Business_Economics.pdf`. Use these as a baseline — you may challenge or refine them with justification.

- Monthly fixed costs: kitchen rent, equipment EMI, utilities, staff salaries
- COGS per pizza: ingredient cost, packaging, delivery cost per order — broken down by pizza type
- Gross margin per pizza and gross margin % by category (base/pizza/topping split)
- Revenue model: average order value, daily orders (weekday vs weekend), monthly revenue at 60% capacity
- Contribution margin analysis: variable cost per order vs revenue per order
- Break-even analysis: minimum daily orders to cover fixed costs
- Delivery radius economics: max radius given rider cost and target delivery time
- GST treatment: how 18% GST flows through the P&L — who absorbs it?

#### Stage 1 Rubric

| Component                       | What We Evaluate                                                                                                            | Pts |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-----|
| PRD completeness                | All functional + non-functional requirements documented, user flow diagram included, written precisely enough to build from | 8   |
| Drawbacks + cost-value analysis | Honest assessment of limitations, not just positives. Cost estimate is credible.                                            | 4   |
| Business unit economics         | Realistic numbers, COGS and margins calculated, break-even shown, GST treatment correct                                     | 8   |

---

|       |  |    |
|-------|--|----|
| TOTAL |  | 20 |
|-------|--|----|

## STAGE 2

### Working MVP on Gradio

Due: Thursday, June 27 · 30 Points · Submit .py + 3 .txt files + sample orders log

Build a fully functional pizza ordering system as a Gradio application in Python. This is your MVP — correctness and robustness matter far more than aesthetics. The system must run end-to-end without crashing on any valid or invalid input.

#### Core Requirements

##### Customer Intake

- Collect customer name: alphabets and spaces only, minimum 2 characters, maximum 40 characters.
- Collect phone number: exactly 10 digits, must start with 6, 7, 8, or 9 (Indian mobile format).
- On invalid input: show a specific, helpful error message and re-prompt. Do not crash.
- Log the timestamp of each new session.

##### Pizza Quantity

- Accept integers from 1 to 10 only. Reject floats, strings, zero, and negatives.
- Apply a 10% discount automatically when quantity  $\geq 5$ . Show the discount line on the bill.
- Maximum outlet capacity is 10 pizzas per order — reject any value above this with an explanation.

##### Menu Selection — loaded from files

- Load Types\_of\_Base.txt, Types\_of\_Pizza.txt, Types\_of\_Toppings.txt at startup.
- Display items as a numbered list with names and prices in INR.
- Accept selection by item number only. Reject out-of-range numbers, letters, and empty input.
- If a file is missing or malformed, display a clear error and exit gracefully.

##### Order Summary and Bill

- Display an itemised bill: Base + Pizza + Topping (per unit), quantity, unit price, subtotal.
- Show discount amount (if applicable), GST @ 18% on the post-discount total, and final payable.
- Bill must be formatted cleanly — aligned columns, currency symbol, totals clearly marked.

##### Payment Flow

- Offer exactly three modes: 1. Cash 2. Card 3. UPI.
- Confirm the selected payment mode. Display an appropriate confirmation message per mode.
- Reject invalid payment selections with a prompt to retry.

##### Order Persistence

- Append every completed order to orders\_log.txt in a consistent, parseable format.
- Each record must include: timestamp, customer name, phone, item selections, unit prices, quantity, subtotal, discount, GST, final total, payment mode.
- Format: one order per block, pipe-separated fields within a line, blank line between orders.

#### EDGE CASES THAT WILL BE TESTED — all must be handled without an unhandled exception:

|                                                      |                                                             |
|------------------------------------------------------|-------------------------------------------------------------|
| (1) Name with only spaces.                           | (5) Entering a price number instead of item number.         |
| (2) Phone number with 10 digits but starting with 1. | (6) Empty input at every prompt.                            |
| (3) Quantity = 0 and quantity = 11.                  | (7) Non-integer at quantity prompt (e.g. "three" or "2.5"). |

(4) Item selection = 0 or greater than menu length.

(8) Running the program with a menu file that has a missing price field.

## Gradio UI Guidelines

Your Gradio app should present the ordering flow as a sequence of steps — not a single giant form. Consider using `gr.Blocks` with a state-driven flow, or a chatbot-style interface. The bill summary must be rendered in a `gr.Dataframe` or `gr.HTML` component — not a plain text box.

## Stage 2 Rubric

| Component                           | What We Evaluate                                                                                         | Pts       |
|-------------------------------------|----------------------------------------------------------------------------------------------------------|-----------|
| Input validation and error handling | All 8 listed edge cases handled gracefully. No unhandled exceptions under any input.                     | 8         |
| Menu file loading                   | All 3 files loaded at runtime. Handles missing files and malformed lines. Works when grader swaps files. | 6         |
| Pricing and discount logic          | Discount threshold correct, GST calculated on post-discount total, itemised bill accurate.               | 7         |
| Payment flow                        | 3 modes offered, invalid mode rejected, confirmation message shown.                                      | 4         |
| Order logging                       | Parseable format, all required fields present, correctly appended on each run.                           | 5         |
| <b>TOTAL</b>                        |                                                                                                          | <b>30</b> |

## STAGE 3

### Full-Stack App + Live Demo

*Due: Wednesday, July 2 · 50 Points · Live presentation in class*

Take your Gradio MVP and rebuild it as a production-grade web application. Frontend deployed on Vercel. Backend and database on Supabase. Add at least one AI feature powered by OpenRouter. Ship it live — a working public URL is mandatory.

#### Technical Stack — All Mandatory

##### Frontend on Vercel

Build a clean, responsive ordering UI. Recommended: Next.js or React. Must be deployed on Vercel with a public URL that works on the day of the demo. The UI must support the full ordering flow — from customer intake to bill display.

##### Backend + Database on Supabase

All orders saved to a Supabase PostgreSQL table. Menu items stored in DB tables — not text files. Schema must have separate tables for menus, orders, and order line items. Use Supabase JS client or REST API from the frontend.

##### Admin Dashboard

Admin login via Supabase Auth. Post-login view: all orders with filters by date and payment mode, total revenue summary, top-selling pizza, busiest hour of the day. Admin can export orders as CSV.

##### Full Stage 2 Logic Preserved

All validation rules, discount thresholds, GST calculation, payment modes, and order summary must work exactly as specified in Stage 2. The full-stack version is not a UI refresh — it is a production deployment of the same system.

#### AI Feature — Pick at Least One

Must be powered by OpenRouter API. System prompt must be documented in your README.

##### Option A — AI Recommendation Engine

After the customer enters their name and phone number, query Supabase for their past orders. Send the order history to an LLM via OpenRouter and ask it to recommend a pizza + topping combination with a short explanation. Show this as a personalised suggestion before the menu selection step. Document your system prompt and the model you chose.

##### Option B — Conversational Ordering Interface

Replace the form-based flow entirely with a chat interface. The customer places their order by typing naturally. An LLM agent (via OpenRouter) extracts the required fields — name, phone, quantity, base, pizza, topping, payment mode — confirms each field, and finalises the order. The agent must handle ambiguity ("something spicy") and re-prompt for missing fields. All business logic (validation, pricing, GST) still applies.

##### Option C — Demand Forecasting Dashboard

Use the order history stored in Supabase to train a lightweight ML model in Python (scikit-learn: LinearRegression or RandomForestRegressor) that predicts order volume by hour of day and day of week. Display the forecast on the admin dashboard as a chart. Show top 3 predicted peak hours for the next 7 days. Document your model choice, features, and evaluation metric (RMSE).

**During the demo, graders will ask each team member to:**

- (1) explain a randomly selected function in your codebase
- (2) walk through one Supabase table schema and justify the design
- (3) modify a live feature — e.g. change the discount threshold from 5 to 3 pizzas and show the result

**Inability to explain or modify any part of the system will result in mark deductions for that individual, regardless of the team score.**

### Submission Checklist — Stage 3

|   |                                                                                                                                            |
|---|--------------------------------------------------------------------------------------------------------------------------------------------|
| ✓ | Public Vercel URL — must be live and functional on July 2                                                                                  |
| ✓ | GitHub repository link — meaningful commit history from all team members across all 3 weeks                                                |
| ✓ | Supabase project URL (read-only access shared with grader)                                                                                 |
| ✓ | README.md with: architecture diagram, setup instructions, AI feature description, system prompt(s) used, OpenRouter model selected and why |
| ✓ | Loom video (3–5 min) walking through every feature — ordering flow, admin dashboard, and AI feature                                        |
| ✓ | Live 10-minute presentation in class — every team member must speak                                                                        |

### Stage 3 Rubric

| Component               | What We Evaluate                                                                             | Pts       |
|-------------------------|----------------------------------------------------------------------------------------------|-----------|
| Vercel frontend — live  | Deployed, responsive, full ordering flow works end-to-end with no crashes                    | 10        |
| Supabase DB integration | Correct schema (3+ tables), orders saved, menu loaded from DB, admin dashboard functional    | 12        |
| Auth + admin dashboard  | Login works, order history with filters, revenue summary, CSV export                         | 8         |
| AI feature              | OpenRouter integrated, system prompt documented in README, feature adds real value to UX     | 12        |
| Live demo + Q&A         | App runs live in class, every team member explains their part, code-level questions answered | 8         |
| <b>TOTAL</b>            |                                                                                              | <b>50</b> |



## 04 · SUBMISSION SUMMARY + RULES OF ENGAGEMENT

### What to Submit and When

| Stage        | What to Submit                                                | Deadline | Pts        |
|--------------|---------------------------------------------------------------|----------|------------|
| Stage 1      | PDF or Notion — PRD + Business Analysis                       | Jun 25   | 20         |
| Stage 2      | Gradio .py file + 3 menu .txt files + sample orders_log.txt   | Jun 27   | 30         |
| Stage 3      | Vercel URL + Supabase access + GitHub repo + Loom + Live Demo | Jul 2    | 50         |
| <b>TOTAL</b> |                                                               |          | <b>100</b> |

### Rules

|   |                                                                                                                                           |
|---|-------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Late submissions lose 5 points per day. Stage 3 cannot be submitted after July 2 — a missing live demo scores zero on the demo component. |
| 2 | GitHub must show commits from all team members. A single upload on the final day scores zero on version control.                          |
| 3 | The grader will swap the menu .txt files before evaluating Stage 2. Build for files you have not seen.                                    |
| 4 | No two teams may submit the same AI feature implementation. First team to commit a feature on GitHub owns it — coordinate early.          |
| ★ | Bonus: implement more than one AI feature and document both — up to +10 bonus points.                                                     |

FDE Programme · · PizzaFlow Applied Project